In [1]:
```
import fastf1
import pandas as pd
import numpy as np

# use the data stored locally
fastf1.Cache.enable_cache('../data/raw')

# load the race - bahrain
session = fastf1.get_session(2025, 4, 'R')
session.load()

laps = session.laps
clean_laps = laps.copy()

# drop incomplete/inaccurate laps - safety car laps, pit laps
clean_laps = clean_laps[clean_laps['IsAccurate'] == True]
clean_laps = clean_laps[clean_laps['PitInTime'].isnull() & clean_laps['PitOu
clean_laps = clean_laps[~clean_laps['TrackStatus'].str.contains('4|6', na=Fa

# create Stint_ID - Cross-Validation Grouping Key
# a 'STINT' is a continuous run on one set of tires
# combine the Driver's 3-letter code with their integer Stint number
clean_laps['Stint_ID'] = clean_laps['Driver'] + "_Stint_" + clean_laps['Stir

# define the target variable
clean_laps['Target_Compound'] = clean_laps['Compound']



# convert LapTime from ns to s
clean_laps['LapTime_Sec'] = clean_laps['LapTime'].dt.total_seconds()

# calc PACE BASELINE: fastest lap time recorded on THAT SPECIFIC LapNumber a
field_best_per_lap = clean_laps.groupby('LapNumber')['LapTime_Sec'].min().re
field_best_per_lap.rename(columns={'LapTime_Sec': 'Field_Best_Lap_Sec'}, inp

# merge baseline back into clean_laps dataframe
clean_laps = clean_laps.merge(field_best_per_lap, on='LapNumber', how='left'

# calc DELTA: how much slower was this driver compared to the fastest car on
clean_laps['Delta_to_Field_Best'] = clean_laps['LapTime_Sec'] - clean_laps['

print(f"Original laps: {len(laps)} | Clean laps remaining: {len(clean_laps)}
print(clean_laps.head())

################################################################### TELEMTR

telemetry_features = []

# loop through clean laps to get the high-frequency telemetry
for index, lap in clean_laps.iterrows():

    try:
        tel = lap.get_telemetry()
```

```
    except Exception as e:
        continue

    # calc basic aggregations
    avg_speed = tel['Speed'].mean()
    std_dev_speed = tel['Speed'].std()
    avg_rpm = tel['RPM'].mean()
    avg_throttle = tel['Throttle'].mean()

    percent_full_throttle = (tel['Throttle'] >= 99).mean()

    # BRAKE EVENTS = how many times the brake pedal goes from 0 to >0
    brake_transitions = (tel['Brake'] > 0).astype(int).diff()
    brake_events = (brake_transitions == 1).sum()

    # Dirty Air / Traffic Proxy
    # DRS values >= 10 indicate the flap is open
    drs_active = int((tel['DRS'] >= 10).any())

    # add extracted features to the original ones
    telemetry_features.append({
        'Index': index,
        'Avg_Speed': avg_speed,
        'Std_Dev_Speed': std_dev_speed,
        'Avg_RPM': avg_rpm,
        'Avg_Throttle': avg_throttle,
        'Percent_Full_Throttle': percent_full_throttle,
        'Brake_Events': brake_events,
        'DRS_Active': drs_active
    })

# convert to dataframe object
tel_df = pd.DataFrame(telemetry_features)
tel_df.set_index('Index', inplace=True)

# merge telemetry features back into clean_laps DataFrame
clean_laps = clean_laps.join(tel_df)

# drop rows where telemetry was missing
clean_laps.dropna(subset=['Avg_Speed'], inplace=True)

# estimate fuel mass feature assuming linear burn rate
clean_laps['Estimated_Fuel_Mass'] = 110.0 - (clean_laps['LapNumber'] * 1.5)

# Degradation Derivative with 3-lap rolling average of the pace delta
clean_laps['Degradation_Slope'] = (
    clean_laps.groupby('Stint_ID')['Delta_to_Field_Best']
    .transform(lambda x: x.rolling(window=3, min_periods=1).mean())
)

# Weather data needs to be sorted by time for an 'asof' merge = merge by nea
weather_data = session.weather_data
weather_data = weather_data.sort_values(by='Time')
clean_laps = clean_laps.sort_values(by='LapStartTime')

weather_subset = weather_data[['Time', 'TrackTemp', 'AirTemp']].rename(colum
```

```
clean_laps = pd.merge_asof(
    clean_laps,
    weather_subset,
    # weather_data[['Time', 'TrackTemp', 'AirTemp']],
    left_on='LapStartTime',
    right_on='Weather_Time',
    direction='backward'
)

clean_laps.drop(columns=['Weather_Time'], inplace=True)

print(f"database final shape: {clean_laps.shape}")
print(clean_laps.columns)
```

```
core           INFO     Loading data for Bahrain Grand Prix - Race [v3.8.1]
req            INFO     Using cached data for session_info
req            INFO     Using cached data for driver_info
req            INFO     Using cached data for session_status_data
req            INFO     Using cached data for lap_count
req            INFO     Using cached data for track_status_data
req            INFO     Using cached data for _extended_timing_data
req            INFO     Using cached data for timing_app_data
core           INFO     Processing timing data...
req            INFO     Using cached data for car_data
req            INFO     Using cached data for position_data
req            INFO     Using cached data for weather_data
req            INFO     Using cached data for race_control_messages
core           INFO     Finished loading data for 20 drivers: ['81', '63',
'4', '16', '44', '1', '10', '31', '22', '87', '12', '23', '6', '7', '14', '3
0', '18', '5', '55', '27']
```

```
Original laps: 1128 | Clean laps remaining: 952
                    Time Driver DriverNumber                LapTime  \
0 0 days 00:58:51.319000    PIA           81 0 days 00:01:37.492000
1 0 days 01:00:29.402000    PIA           81 0 days 00:01:38.083000
2 0 days 01:02:07.535000    PIA           81 0 days 00:01:38.133000
3 0 days 01:03:45.578000    PIA           81 0 days 00:01:38.043000
4 0 days 01:05:23.941000    PIA           81 0 days 00:01:38.363000

   LapNumber  Stint PitOutTime PitInTime            Sector1Time  \
0        2.0    1.0        NaT       NaT 0 days 00:00:31.139000
1        3.0    1.0        NaT       NaT 0 days 00:00:31.306000
2        4.0    1.0        NaT       NaT 0 days 00:00:31.326000
3        5.0    1.0        NaT       NaT 0 days 00:00:31.305000
4        6.0    1.0        NaT       NaT 0 days 00:00:31.336000

             Sector2Time  ... Position Deleted DeletedReason FastF1Generated
\
0 0 days 00:00:42.343000  ...      1.0   False                         False
1 0 days 00:00:42.727000  ...      1.0   False                         False
2 0 days 00:00:42.796000  ...      1.0   False                         False
3 0 days 00:00:42.690000  ...      1.0   False                         False
4 0 days 00:00:42.880000  ...      1.0   False                         False

   IsAccurate     Stint_ID  Target_Compound  LapTime_Sec  Field_Best_Lap_Sec
\
0        True  PIA_Stint_1             SOFT       97.492              97.492
1        True  PIA_Stint_1             SOFT       98.083              98.083
2        True  PIA_Stint_1             SOFT       98.133              98.133
3        True  PIA_Stint_1             SOFT       98.043              98.043
4        True  PIA_Stint_1             SOFT       98.363              98.363

  Delta_to_Field_Best
0                 0.0
1                 0.0
2                 0.0
3                 0.0
4                 0.0

[5 rows x 36 columns]
database final shape: (952, 47)
Index(['Time', 'Driver', 'DriverNumber', 'LapTime', 'LapNumber', 'Stint',
       'PitOutTime', 'PitInTime', 'Sector1Time', 'Sector2Time', 'Sector3Tim
e',
       'Sector1SessionTime', 'Sector2SessionTime', 'Sector3SessionTime',
       'SpeedI1', 'SpeedI2', 'SpeedFL', 'SpeedST', 'IsPersonalBest',
       'Compound', 'TyreLife', 'FreshTyre', 'Team', 'LapStartTime',
       'LapStartDate', 'TrackStatus', 'Position', 'Deleted', 'DeletedReaso
n',
       'FastF1Generated', 'IsAccurate', 'Stint_ID', 'Target_Compound',
       'LapTime_Sec', 'Field_Best_Lap_Sec', 'Delta_to_Field_Best', 'Avg_Spee
d',
       'Std_Dev_Speed', 'Avg_RPM', 'Avg_Throttle', 'Percent_Full_Throttle',
       'Brake_Events', 'DRS_Active', 'Estimated_Fuel_Mass',
       'Degradation_Slope', 'TrackTemp', 'AirTemp'],
      dtype='object')
```

In [2]:
```
# choose features to keep
features = [
    'Delta_to_Field_Best', 'Avg_Speed', 'Std_Dev_Speed', 'Avg_RPM',
    'Avg_Throttle', 'Percent_Full_Throttle', 'Brake_Events', 'DRS_Active',
    'Estimated_Fuel_Mass', 'Degradation_Slope', 'TrackTemp', 'TyreLife'
]

metadata = ['Target_Compound', 'Stint_ID']

model_df = clean_laps[metadata + features].copy()

# drop NaN values
model_df.dropna(inplace=True)

print(f"final modeling database shape: {model_df.shape}")
print(model_df.columns)
```

```
final modeling database shape: (952, 14)
Index(['Target_Compound', 'Stint_ID', 'Delta_to_Field_Best', 'Avg_Speed',
       'Std_Dev_Speed', 'Avg_RPM', 'Avg_Throttle', 'Percent_Full_Throttle',
       'Brake_Events', 'DRS_Active', 'Estimated_Fuel_Mass',
       'Degradation_Slope', 'TrackTemp', 'TyreLife'],
      dtype='object')
```

In [3]:
```
############################################ DATABASE MODELING VERIFICATION
import os

# check data types and missing values
print(model_df.info())
print("-" * 30)
print("Missing values per column:\n", model_df.isnull().sum())
print("-" * 30)

print(model_df[['Percent_Full_Throttle', 'DRS_Active', 'Delta_to_Field_Best'

os.makedirs('../data/cleaned', exist_ok=True)
model_df.to_csv('../data/cleaned/model_df.csv', index=False)
```

```
<class 'fastf1.core.Laps'>
RangeIndex: 952 entries, 0 to 951
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Target_Compound       952 non-null    object
 1   Stint_ID              952 non-null    object
 2   Delta_to_Field_Best   952 non-null    float64
 3   Avg_Speed             952 non-null    float64
 4   Std_Dev_Speed         952 non-null    float64
 5   Avg_RPM               952 non-null    float64
 6   Avg_Throttle          952 non-null    float64
 7   Percent_Full_Throttle 952 non-null    float64
 8   Brake_Events          952 non-null    int64
 9   DRS_Active            952 non-null    int64
 10  Estimated_Fuel_Mass   952 non-null    float64
 11  Degradation_Slope     952 non-null    float64
 12  TrackTemp             952 non-null    float64
 13  TyreLife              952 non-null    float64
dtypes: float64(10), int64(2), object(2)
memory usage: 104.3+ KB
None
------------------------------
Missing values per column:
 Target_Compound         0
Stint_ID                0
Delta_to_Field_Best     0
Avg_Speed               0
Std_Dev_Speed           0
Avg_RPM                 0
Avg_Throttle            0
Percent_Full_Throttle   0
Brake_Events            0
DRS_Active              0
Estimated_Fuel_Mass     0
Degradation_Slope       0
TrackTemp               0
TyreLife                0
dtype: int64
------------------------------
       Percent_Full_Throttle  DRS_Active  Delta_to_Field_Best  \
count             952.000000  952.000000           952.000000
mean                0.451500    0.432773             1.573539
std                 0.031285    0.495720             0.945470
min                 0.309237    0.000000             0.000000
25%                 0.431315    0.000000             0.897750
50%                 0.451393    0.000000             1.556500
75%                 0.471002    1.000000             2.226250
max                 0.554534    1.000000             5.693000

       Estimated_Fuel_Mass
count           952.000000
mean             66.178571
std              25.067556
min              24.500000
25%              44.000000
```

```
50%              68.000000
75%              87.500000
max             107.000000
```

In [10]:

```
################################################################ CROSS-VALIDA

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedGroupKFold
from sklearn.metrics import f1_score
from sklearn.utils.class_weight import compute_sample_weight
import xgboost as xgb
import catboost as cb
import lightgbm as gbm
import optuna
import numpy as np

# CUSTOM WEIGHTS FOR THE COMPOUNDS
custom_weights = {0: 1.5, 1: 3.5, 2: 1.0}

# encode target variables: hard, medium, soft -> 0, 1, 2
le = LabelEncoder()
model_df['Target_Encoded'] = le.fit_transform(model_df['Target_Compound'])
label_mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print(f"Class mapping: {label_mapping}")

features = [
    'Delta_to_Field_Best', 'Avg_Speed', 'Std_Dev_Speed', 'Avg_RPM',
    'Avg_Throttle', 'Percent_Full_Throttle', 'Brake_Events', 'DRS_Active',
    'Estimated_Fuel_Mass', 'Degradation_Slope', 'TrackTemp', 'TyreLife'
]

X = model_df[features]
y = model_df['Target_Encoded']
groups = model_df['Stint_ID']


# define the Optuna obj function for XGBOOST
def obj_xgboost(trial):

    param = {
        'objective': 'multi:softmax',
        'num_class': 3,
        'tree_method': 'hist',
        'random_state': 42,
        'max_depth': trial.suggest_int('max_depth', 3, 7),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log
        'subsample': trial.suggest_float('subsample', 0.5, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0
        'min_child_weight': trial.suggest_int('min_child_weight', 1, 15),
        'gamma': trial.suggest_float('gamma', 1e-8, 1.0, log=True),
        'lambda': trial.suggest_float('lambda', 1e-3, 10.0, log=True),
        'alpha': trial.suggest_float('alpha', 1e-3, 10.0, log=True)
    }

    # use StratifiedGroupKFold to prevent stint leakage
    # all laps from a single STINT_ID stay together in either training or va
```

```
    sgkf = StratifiedGroupKFold(n_splits=5)
    f1_scores = []

    for train_idx, val_idx in sgkf.split(X, y, groups):
        X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
        y_train, y_val = y.iloc[train_idx], y.iloc[val_idx]

        # sample_weights = compute_sample_weight(class_weight=custom_weights
        sample_weights = compute_sample_weight(class_weight='balanced', y=y_

        # initialize and train XGBoost classifier model
        # use dynamic sample weights for class imbalances
        model = xgb.XGBClassifier(**param)
        model.fit(
            X_train, y_train,
            sample_weight=sample_weights,
            eval_set=[(X_val, y_val)],
            verbose=False
        )

        # predict and evaluate using Macro F1-Score
        preds = model.predict(X_val)
        fold_f1 = f1_score(y_val, preds, average='macro')
        f1_scores.append(fold_f1)

    return np.mean(f1_scores)

optuna.logging.set_verbosity(optuna.logging.WARNING)

study = optuna.create_study(direction='maximize')
study.optimize(obj_xgboost, n_trials=150)

print(f"Best Macro F1-Score: {study.best_trial.value:.4f}")
print(f"Best Parameters: {study.best_trial.params}")

# train final model on the best parameters found
best_params = study.best_trial.params
best_params['objective'] = 'multi:softmax'
best_params['num_class'] = 3
best_params['random_state'] = 42


final_model = xgb.XGBClassifier(**best_params)

# global_sample_weights = compute_sample_weight(class_weight=custom_weights,
global_sample_weights = compute_sample_weight(class_weight='balanced', y=y)

final_model.fit(X, y, sample_weight=global_sample_weights)
```

```
Class mapping: {'HARD': np.int64(0), 'MEDIUM': np.int64(1), 'SOFT': np.int64
(2)}
Best Macro F1-Score: 0.6451
Best Parameters: {'max_depth': 5, 'learning_rate': 0.09010366816935095, 'sub
sample': 0.8608565787440378, 'colsample_bytree': 0.7128343242901627, 'min_ch
ild_weight': 14, 'gamma': 5.539373496867535e-07, 'lambda': 4.10423028806846
8, 'alpha': 0.0065448140632123515}
```

Out[10]:

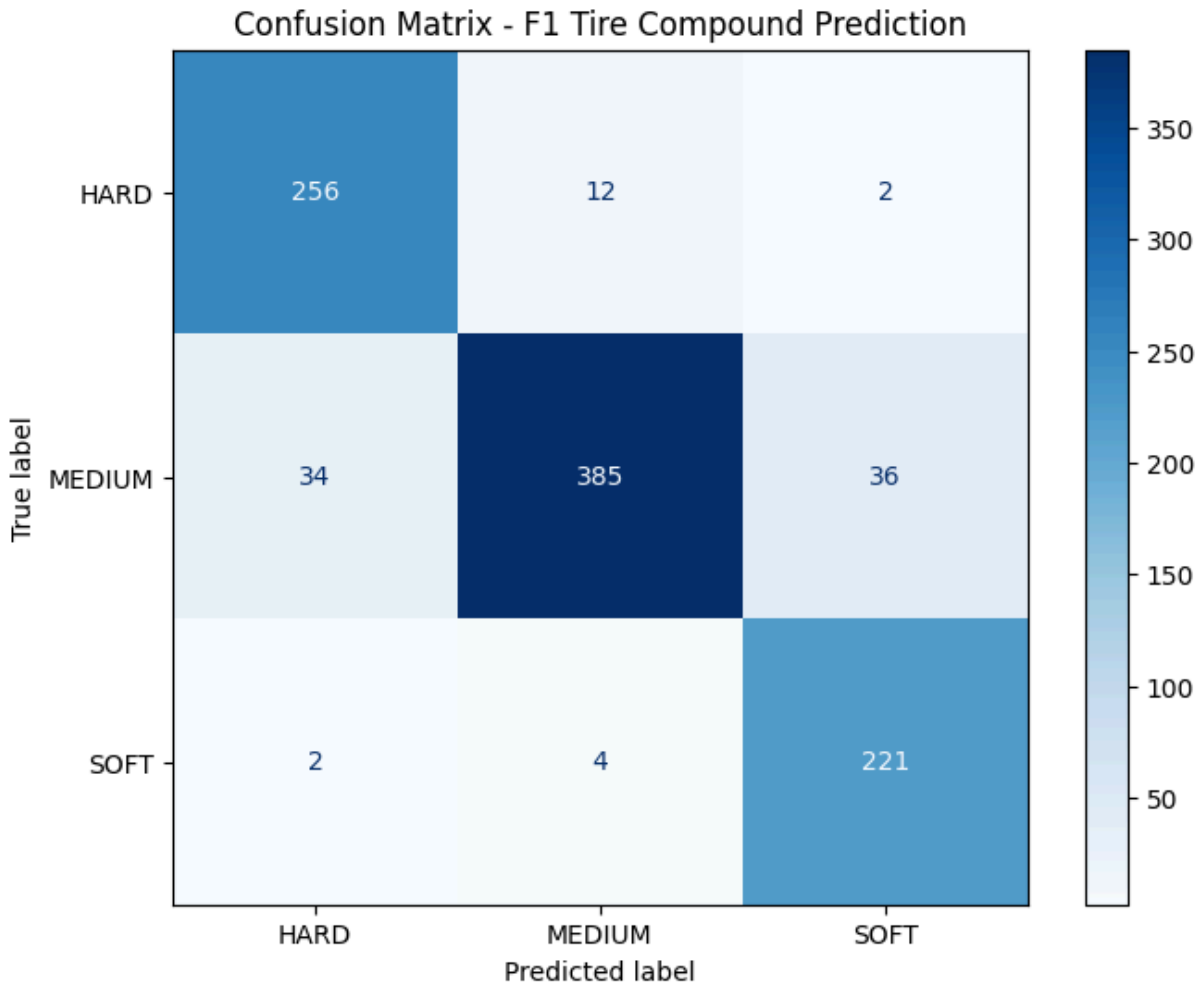
▾ XGBClassifier

▸ Parameters

In [11]:

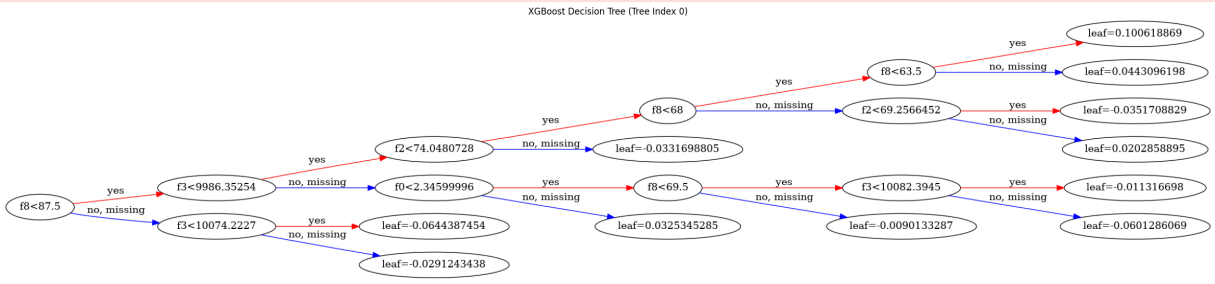
```
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix, Confusi
import shap
import xgboost as xgb

##################################################################### EVALUA

y_pred = final_model.predict(X)

target_names = ['HARD', 'MEDIUM', 'SOFT']
print(classification_report(y, y_pred, target_names=target_names))

# CONFUSION MATRIX
print("CONFUSION MATRIX")
cm = confusion_matrix(y, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=target_nam

fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(cmap='Blues', ax=ax)
plt.title("Confusion Matrix - F1 Tire Compound Prediction")
plt.show()


# XGBoost Tree Visualization
print("XGBOOST DECISION TREE")
fig, ax = plt.subplots(figsize=(30, 15))
xgb.plot_tree(final_model, num_trees=0, ax=ax, rankdir='LR')
plt.title("XGBoost Decision Tree (Tree Index 0)")
plt.show()


# SHAP Interpretability Plot
print("SHAP PLOT")
explainer = shap.TreeExplainer(final_model)
shap_values = explainer.shap_values(X)
shap.summary_plot(shap_values, X, class_names=target_names, plot_type="bar",
plt.show()
```

```
              precision    recall  f1-score   support

        HARD       0.88      0.95      0.91       270
      MEDIUM       0.96      0.85      0.90       455
        SOFT       0.85      0.97      0.91       227

    accuracy                           0.91       952
   macro avg       0.90      0.92      0.91       952
weighted avg       0.91      0.91      0.91       952

CONFUSION MATRIX
```

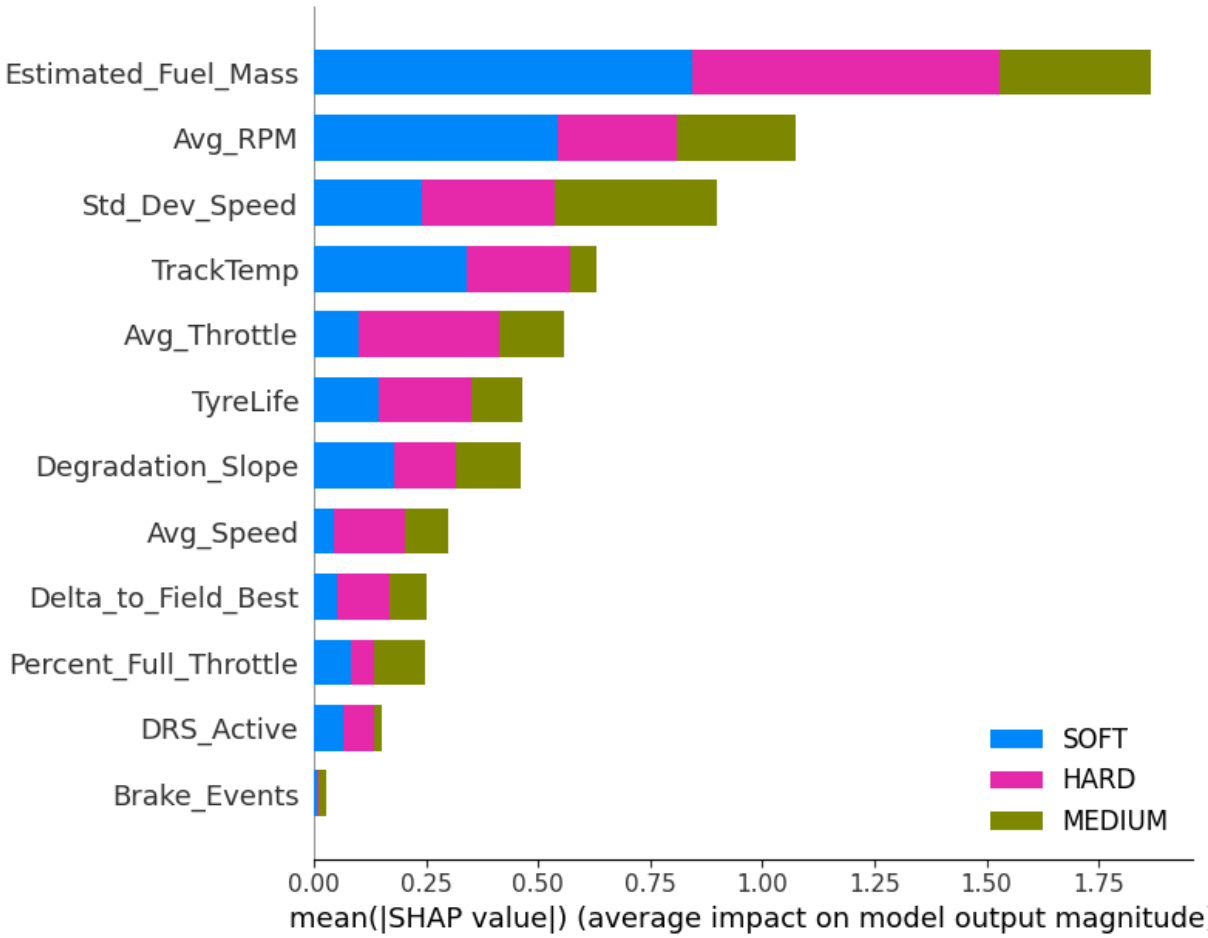


XGBOOST DECISION TREE

```
/home/c/Desktop/AI_MUAR/ai_f1/.venv/lib/python3.13/site-packages/xgboost/plo
tting.py:268: FutureWarning: The `num_trees` parameter is deprecated, use `t
ree_idx` insetad.
  warnings.warn(
```



SHAP PLOT

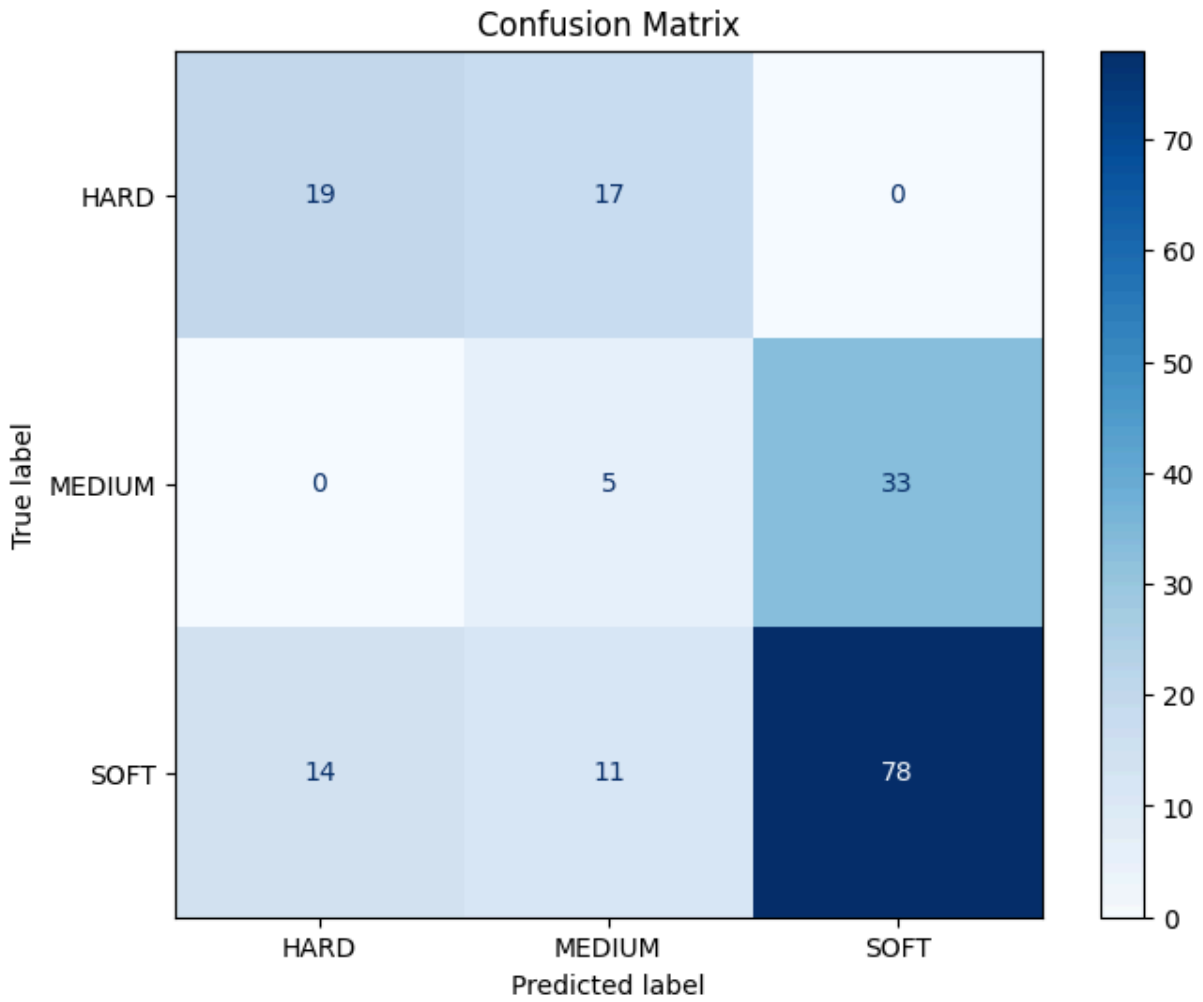


In [ ]:

In [12]:
```
from sklearn.model_selection import GroupShuffleSplit
from sklearn.metrics import classification_report, confusion_matrix, Confusi
from sklearn.utils.class_weight import compute_sample_weight
import xgboost as xgb
import shap
import matplotlib.pyplot as plt

################################################################### STRICT

# set up grouped split - 80% Train, 20% Test
gss = GroupShuffleSplit(test_size=0.20, n_splits=1, random_state=42)
train_idx, test_idx = next(gss.split(X, y, groups=groups))

X_train_strict = X.iloc[train_idx]
X_test_strict = X.iloc[test_idx]
y_train_strict = y.iloc[train_idx]
y_test_strict = y.iloc[test_idx]

print(f"TRAININING on {len(X_train_strict)} laps. TESTING on {len(X_test_str

# calc sample weights ONLY on training data
# train_sample_weights = compute_sample_weight(class_weight=custom_weights,
train_sample_weights = compute_sample_weight(class_weight='balanced', y=y_tr
```

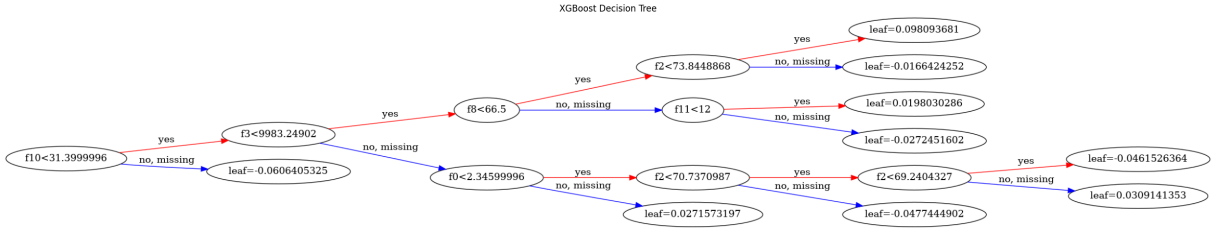
```
# initialize and train the model ONLY on the 80% training data
strict_model = xgb.XGBClassifier(**best_params)
strict_model.fit(X_train_strict, y_train_strict, sample_weight=train_sample_

# predict ONLY on the 20% holdout test data
y_pred_strict = strict_model.predict(X_test_strict)

print("\n TRUE UNSEEN CLASSIFICATION REPORT")
print(classification_report(y_test_strict, y_pred_strict, target_names=targe

print("\n CONFUSION MATRIX")
cm_strict = confusion_matrix(y_test_strict, y_pred_strict)
disp_strict = ConfusionMatrixDisplay(confusion_matrix=cm_strict, display_lab

fig, ax = plt.subplots(figsize=(8, 6))
disp_strict.plot(cmap='Blues', ax=ax)
plt.title("Confusion Matrix")
plt.show()

print("\n XGBOOST DECISION TREE")
fig, ax = plt.subplots(figsize=(30, 15))
xgb.plot_tree(strict_model, num_trees=0, ax=ax, rankdir='LR')
plt.title("XGBoost Decision Tree")
plt.show()

print("\n SHAP INTERPRETABILITY")
explainer_strict = shap.TreeExplainer(strict_model)
shap_values_strict = explainer_strict.shap_values(X_test_strict)

shap.summary_plot(shap_values_strict, X_test_strict, class_names=target_name
```

```
TRAININING on 775 laps. TESTING on 177 unseen laps.

 TRUE UNSEEN CLASSIFICATION REPORT
              precision    recall  f1-score   support

        HARD       0.58      0.53      0.55        36
      MEDIUM       0.15      0.13      0.14        38
        SOFT       0.70      0.76      0.73       103

    accuracy                           0.58       177
   macro avg       0.48      0.47      0.47       177
weighted avg       0.56      0.58      0.57       177


 CONFUSION MATRIX
```

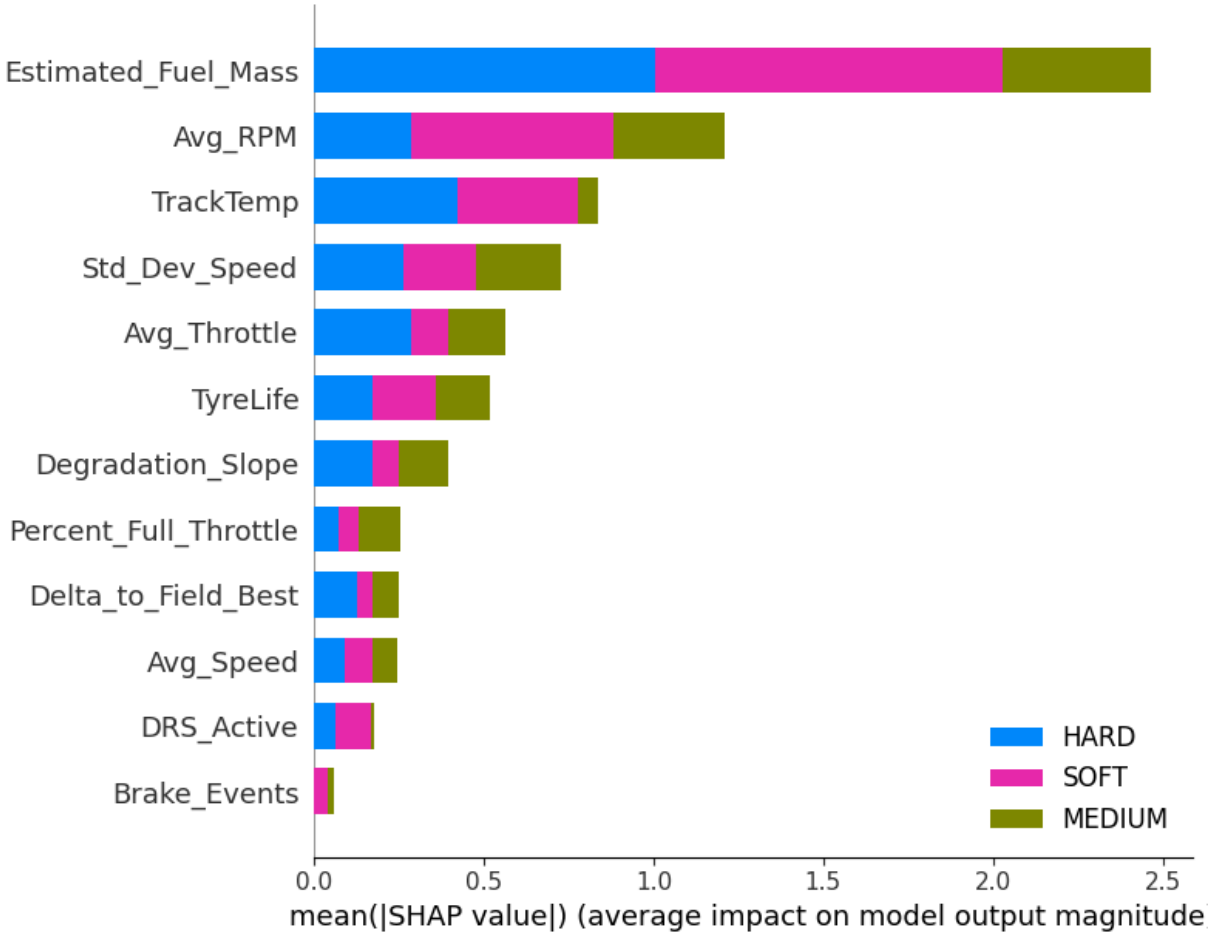


XGBOOST DECISION TREE

```
/home/c/Desktop/AI_MUAR/ai_f1/.venv/lib/python3.13/site-packages/xgboost/plo
tting.py:268: FutureWarning: The `num_trees` parameter is deprecated, use `t
ree_idx` insetad.
  warnings.warn(
```



SHAP INTERPRETABILITY


Estimated_Fuel_Mass
Avg_RPM
TrackTemp
Std_Dev_Speed
Avg_Throttle
TyreLife
Degradation_Slope
Percent_Full_Throttle
Delta_to_Field_Best
Avg_Speed
DRS_Active
Brake_Events
HARD
SOFT
MEDIUM
0.0
0.5
1.0
1.5
2.0
2.5
mean(|SHAP value|) (average impact on model output magnitude

In [ ]: